



نظري

كلية الهندسة المعلوماتية

السنة الثانية

Java Swing:
Extra content

عمل طلابي



RB Informatics; 8/1/2025

برمجة ٣

مرحباً بزملائنا! نذكركم بأن محتوى محاضرات الاكسترا هو محتوى طلابي إثرائي لا يغني عن المحاضرات الأساسية، وليس مطلوباً لامتحان، بالتوفيق!

WHAT IS SWING?

هي GUI Toolkit ضمن جافا تتكون من مجموعة من الكلاسات التي يمكننا من خلالها إنشاء واجهة رسومية تفاعلية يمكن للمستخدم التعامل معها و هي امتداد لمكتبة سابقة تدعى AWT ، حيث أضافت عليها العديد من العناصر وخصائص متقدمة و طورت من أداؤها.

من أهم ميزاتها:

أسرع وأخف على الجهاز مقارنة بـ AWT لأن:

■ Swing مكتوبة بلغة Java مما يجعلها مستقلة عن نظام التشغيل (تقوم بتكوين واجهاتها بشكل مستقل باستخدام JVM مما يجعلها متوافقة عبر الأنظمة المختلفة)

■ AWT مكتوبة بلغة C مما يعني أنها تعتمد على نظام التشغيل أي تبحث عن كيفية تكوين المكونات الرسومية داخل نظام التشغيل نفسه.

بشكل عام تعتمد swing في تكوين الواجهة على ما نسميه بال components وكل component يمثل أحد الأمرين إما container (حاوية) أو item (عنصر) .
تصميم الواجهة يعتمد على توضع هذه العناصر ضمن هذه الحاويات لتظهر بشكل معين أمام المستخدم ويمكن كذلك وضع حاويات ضمن حاويات وهكذا بشكل يظهر لنا تصميماً معيناً .

جميع هذه المكونات ترث من JComponent

من الأمثلة على components ضمن swing :

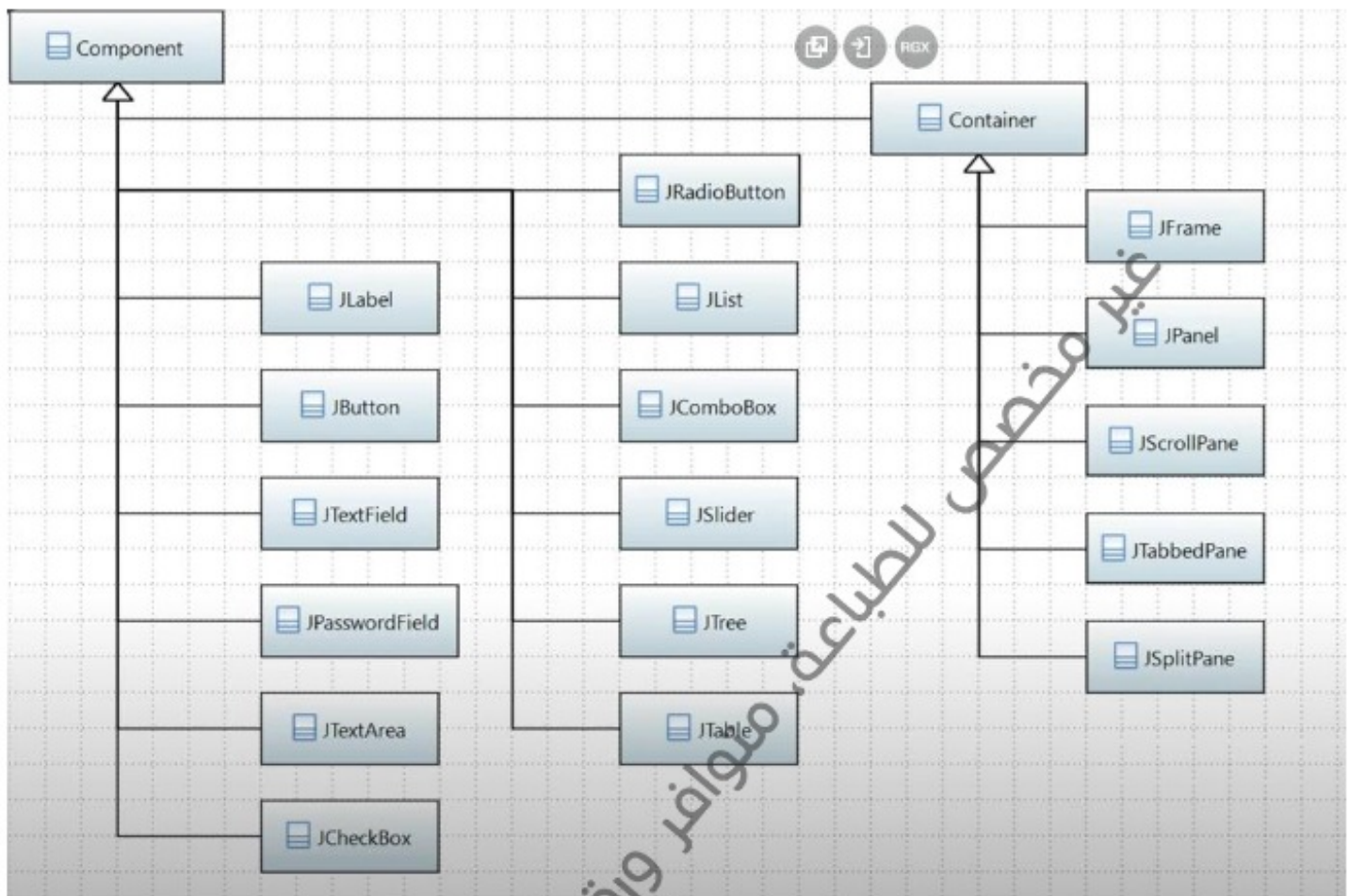
- JButton (عنصر) يظهر زر
- JPanel (حاوية) يظهر على شكل واجهة فارغة ضمن نافذة



جنة جنة جنة

جنة يا وطننا...

سوف نبدأ بشرح العناصر تباعاً بصورة واضحة:



هذا المخطط يحوي على المكونات الأكثر استخداماً و الشيع، وسنشرح معظمها.

■ **JFrame**: الكلاس الذي يمثل ال window التي سنتعامل معها و تظهر أمامنا و تحوي برنامجنا و بالتالي يمكن اعتبارها الحاوية الأساسية و سنشرح التوابع الأساسية لهذا ال component

مثال:

```

1 import javax.swing.*;
2 import java.awt.*;
3 public class Main {
4     public static void main(String[] args) {
5         JFrame frame = new JFrame();
6         // تحديد عنوان النافذة
7         frame.setTitle("string");
8 //      إغلاق ال MAIN THREAD عند إغلاق ال EDT THREAD (إغلاق البرنامج عند إغلاق النافذة).
//      النافذة يتم تشغيلها عادة على ثريد مستقلة، هذا يجعل البرنامج كاملاً ينتهي بإغلاق النافذة
  
```

```

9     frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
10    // حجم النافذة
11    frame.setSize(500,500);
12    // تحديد موقع النافذة على الشاشة
13    frame.setLocation(100, 100);
14    // منع تغيير حجم النافذة
15    frame.setResizable(false);
16    // سنقوم بتعريف زر لشرح تابعين مهمين
17    JButton jButton= new JButton();
18    // إضافة عنصر للحاوية
19    frame.add(jButton);
20    // حذف عنصر من الحاوية
21    frame.remove(jButton);
22    // أهم تابع و هو الذي سيظهر النافذة، عدم تفعيله يؤدي لعدم ظهور النافذة
23    frame.setVisible(true);
24 }
25 }

```

■ JPanel : هي كلاس يمثل container يمكن وضع عناصر ضمنه و تصميمه و وضعه ضمن الفريم

مثال:

```

import javax.swing.*;
import java.awt.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setTitle("string");
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        JPanel jPanel = new JPanel();
        // وضع لون بالخلفية
        jPanel.setBackground(new Color(40,40,40));
        // وضع الحجم الأمثل له
        jPanel.setPreferredSize(new Dimension(400,400));
        // وضع layout manager ، سيتم شرحه بفقرة مستقلة وهنا فقط سنظهر الأمر
        // ( يمكن استخدامه للفريم كذلك )
        jPanel.setLayout(null);
    }
}

```

```
JLabel jLabel = new JLabel("test");
// اضافة عنصر
jPanel.add(jLabel);
// حذف عنصر
jPanel.remove(jLabel);
// اعادة تقدير موقع العناصر يدوياً في ال layout بعد إضافة عناصر جديدة عليها (يتم استدعاء
// التابع تلقائياً أحياناً بحالات معينة)
jPanel.revalidate();
// اعادة رسم موقع العناصر يدوياً (يتم استدعاء التابع تلقائياً أحياناً بحالات معينة مع التابع السابق)
// repaint: تستدعي التابع paintComponent ضمنها وأحياناً يتم استدعاؤها تلقائياً، مثلاً عندما نقوم
// بـ minimize للتطبيق ومن ثم نقوم بإعادة تكبيره (فسيتم استدعاء repaint)
jPanel.repaint();
// نضيف البانل للفريم
frame.add(jPanel);
// يمكن إضافة أكثر من panel وستكون الثانية فوق الأولى
frame.add(new JPanel());
frame.setSize(500,500);
frame.setLocation(100, 100);
frame.setResizable(false);
frame.setVisible(true); }
```

■ **JLabel**: أول عنصر سنشرحه و هو يمثل نصاً ثابتاً يمكن إظهاره على الشاشة يمكن كذلك اظهار "صور" من خلاله.

مثال :

```
import javax.swing.*;
import java.awt.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setTitle("string");
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        // الكونستر كتر يمكن وضع نص ابتدائي فيه
        JLabel jLabel = new JLabel("test");
        // تغيير النص
        jLabel.setText("any thing you want");
    }
}
```

```
// تحديد الخط حيث الخطوط معتمدة على المكتبة السابقة (awt)
jLabel.setFont(new Font("Arial",Font.BOLD,30));
// هنا لو أردنا وضع صورة
jLabel.setIcon(new ImageIcon("path of the image if you want"));
// موضع النص الأفقي، وقمنا باختيار المنتصف ؟
jLabel.setHorizontalAlignment(SwingConstants.CENTER);
frame.add(jLabel);
frame.setSize(500,500);
frame.setLocation(100, 100);
frame.setResizable(false);
frame.setVisible(true);
}
}
```

■ **JButton** : ثاني عنصر سنشرحه و هو زر طبعا سيكون فقط شكلياً الآن بالفقرات اللاحقة سنتحدث عن كيف نجعله يعمل

مثال :

```
import javax.swing.*;
import java.awt.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setTitle("string");
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        // الكونسترक्टर يمكن وضع نص ابتدائي فيه يظهر على الزر
        JButton button = new JButton("lol");
        // تغيير النص
        button.setText("Click Me");
        // تحديد موقع و حجم العنصر بالنسبة لحاويته الاحداثيات تبدأ من اذ تكون النقطة 0,0 هي
        // تلك أعلى اليسار
        button.setBounds(0,0,200,200);
        // هنا لو اردنا وضع صورة
        button.setIcon(new ImageIcon("path of the image if you want"));
        // لون خلفية الزر
        button.setBackground(Color.black);
    }
}
```

```
// لون الكتابة التي على الزر
button.setForeground(Color.orange);
// تعطيل او تفعيل الزر
button.setEnabled(true);
// هذا ما سيجعل الزر يعمل نشرحه لاحقاً
button.addActionListener(null);
frame.add(button);
frame.setSize(500,500);
frame.setLocation(100, 100);
frame.setResizable(false);
frame.setVisible(true);
}
}
```

طبعا سنشرح باقي العناصر على عجلة لكي نتقل للفقرة التالية، يوجد العديد من التوابع التي يمكن استخدامها استعرضنا بعضها فقط و في كل عنصر سنشرحه تالياً سنذكر أهم ميزات و التوابع ضمنه لكن دون كود كامل.

■ **JTextField**: حقل لإدخال النص من قبل المستخدم على الواجهة، الكونستركتر يأخذ حجماً عدد خانات النص المدخل، والافراضي هو 0

setColumns: عدد الاعمدة ضمنه (حجمه)

setText: تضع نص افتراضي ضمنه

getText: ترجع النص الموجود ضمنه ك STRING

setEditable: تمنع التعديل او تسمح به

■ **JTextArea**: حقل لإدخال نص من قبل المستخدم على الواجهة لكنه يحوي أكثر من سطر وله نفس التوابع السابقة

■ **JPasswordField**: حقل لإدخال كلمة مرور من قبل المستخدم

getPassword يرجع كلمة المرور ك char array

getText ترجعه ك String

setEchoChar تحدد كيف ستظهر كلمة المرور للمستخدم "نجوم ام حروف او ما شابه " لو مررنا "*" سيظهر نجوم مكان الأحرف و لو مررنا 0 (char) سيظهر الأحرف كما هي

- **JScrollPane** : حاوية لها ScrollBar يظهر في حال زاد حجم العنصر الذي تحتويه عن قدر معين ، نمرر العنصر الذي نود منها احتواءه من خلال الباني أو `setViewport()`

ملاحظة : من المفيد استخدامه مع `JTextArea` مثلا كون النص قابل للتمدد كثيراً

- **JCheckBox** : checkbox صندوق يمكن من خلاله اختيار او عدم اختيار أمر ما
`isSelected` ترجع هل هذا الزر تم تحديده أم لا
`setSelected` تجعل الزر يتم تحديده أو لا يتم تحديده

- **JRadioButton** : كسابقه لكن في هذا يمكن تحديد خيار واحد حصراً من مجموعة خيارات و هذا يتم من خلال استخدام `ButtonGroup` حيث سنجمع فيه أكثر من `radiobutton` من خلال التابع `add`
يمكن استخدام التابعين السابقين `isSelected` and `setSelected`

- **JComboBox** : يظهر قائمة منسدلة من الخيارات يمكن اختيار احدها و هو `generic`، أي:
`JComboBox<datatype> jcomboBox = new JComboBox<>();`



نستخدم `add item` لإضافة عنصر
`getSelectedItem` لإرجاع العنصر المختار
`setSelectedItem` لوضع العنصر المختار
`JList`: تظهر قائمة ظاهرة من العناصر يمكن تحديد أحدها
`setListData`: نمرر لها مصفوفة من البيانات التي نود عرضها
`getSelectedValue`: لإرجاع القيمة التي حددناها

- **JTabbedPane** حاوية فيها عدة نوافذ يمكن التنقل بينها من خلال `tabs` تظهر في أعلاها يمكن إضافة عدة `panels` لها و كل `tab` ستحتوي `panel` مستقلة

هنا سنكتفي بشرح هذا العدد من العناصر يوجد المزيد طبعا و لكن هذا كفيل بشرح الفكرة العامة لعناصر `swing`

لنتنقل الآن لفكرة أخرى و هي كيف يمكن تنظيم و تنسيق العناصر على الشاشة بالصورة التي نبتغيها لأنه حسب ما رأينا نحن فقط نقوم باستخدام `add` ونضيف العنصر للحاوية أو نضيف الحاوية لحاوية أخرى

لكن هل هذا كافٍ؟ لا

لحل هذا الأمر لدينا حلان :

1. إما تحديد موقع العناصر يدوياً من خلال تابع يدعى `setBounds` الذي يأخذ الإحداثيات (x,y) حيث تكون $(0,0)$ هي أعلى نقطة يساراً و الاتجاه الأيمن هو اتجاه الفواصل الموجب و الاتجاه للأسفل هو اتجاه الترتيب الموجب ويأخذ الطول و العرض بحيث يمكننا تحديد مكان العنصر بشكل دقيق على الشاشة.

2. او نستخدم ما يسمى بال `layout manager` و هو عبارة عن كلاسات يمكنها تنسيق الحاويات الخاصة بك بشكل معين بحيث تضع العناصر بشكل معين.

يوجد أكثر من نوع منها سنستعرض بعضها بالتفصيل

Layout Name	Description
FlowLayout	Lays out components in a single row, starting a new row if its container is not sufficiently wide
GridLayout	Makes a bunch of components equal in size and displays them in the requested number of rows and columns
GridBagLayout	Aligns components by placing them within a grid of cells allowing components to span more than one cell. The rows in the grid can have different heights. and grid columns can have different widths.
BorderLayout	Places components in up to five areas: top, bottom, left, right, and center. All extra space is placed in the center area.
BoxLayout	Puts components in a single row or column. It respects the components' requested maximum sizes and also lets you align components
CardLayout	Lets you implement an area that contains different components at different times.

نبدأ بال Layout :

- **FlowLayout** : ببساطة العناصر تتوضع بجانب بعضها وعند الوصول لحدود الحاوية تبدأ سطرًا جديدًا (عند تصغير الشاشة يدويًا مثلًا ستنزل العناصر لأسطر جديدة وتتمدد)

```
frame.setLayout(new FlowLayout());
```

هكذا يتم تحديد ال LAYOUT MANAGER للحاوية

ملاحظة : يمكن كل حاوية امتلاك LAYOUT MANAGER واحد

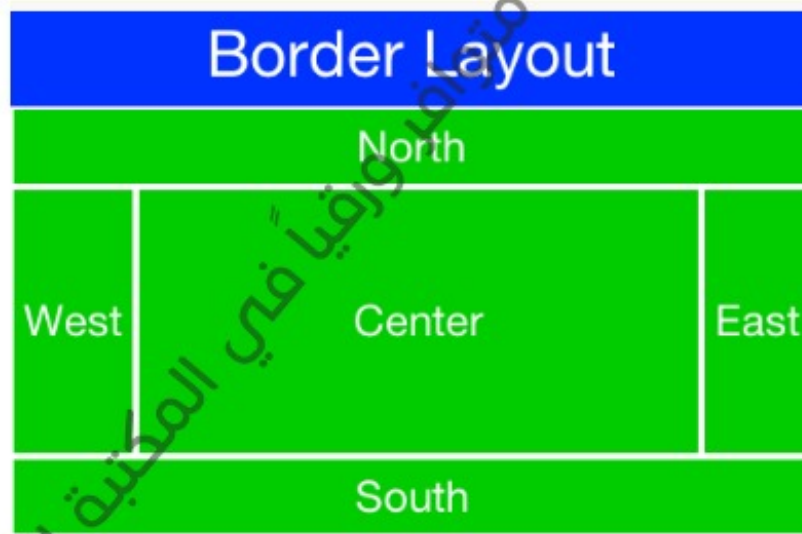
- **BorderLayout** : ببساطة يقسم الحاوية لـ 5 مناطق مبينة بالشكل التالي :

```
frame.setLayout (new BorderLayout());
```

يمكننا توزيع العناصر بالمنطقة التي نريدها من خلال:

```
frame.add (new JButton(),BorderLayout.Center);
```

بهذا سيضاف للوسط و يمكن إضافة ما تشاء من العناصر .



- **GridLayout** : يقسم الحاوية لشبكة من n سطر و m عمود عند إضافة كل عنصر يأخذ مكاناً في هذه الشبكة و يملأه عن طريق التابع

```
frame.setLayout(new GridLayout(n,m));
```

يمكن تمرير بارامترات إضافية لكونستركتور آخر يقوم بعمل grid لكن بفراغات بين العناصر

```
frame.setLayout(new GridLayout (n,m,10,10));
```

* البارامتر الثالث (10) فراغ أفقي

* البارامتر الرابع (10) فراغ عمودي بين العناصر

الرابع و هو مشابه لسابقه:

- GridBagLayout : هو شبكة كذلك لكنه أكثر مرونة حيث أنه ذو حجم ديناميكي و يمكن التحكم بملء العناصر بمكانها بالشبكة و تحديد حجمها نسبةً لهذه الشبكة
- نقوم بهذا من خلال مكون نسميه GridBagConstraints حيث عند إضافة العنصر للحاوية التي يتم تنظيمها من خلال الـ gridbaglayout نمرر هذا المكون معه ،حيث سيحدد كيف سيتوضع العنصر ضمن الشبكة

أما عن ما هي الأمور التي يمكن تحديدها فهي كالتالي :

- gridx,gridy : تحدد مكان العنصر على محور الفواصل و محور الترتيب
- gridheight,gridweight : تحدد الحجم الذي سيشغله العنصر بالعرض او بالطول مقدراً ب (عدد الاعمدة و عدد الاسطر)
- widthx,height : تحدد كمية الحجم الإضافي الذي سيشغله العنصر في حال ازداد حجم الشاشة
- fill في حال كان العنصر اصغر من حجم الخلية التي سيشغلها نحدد من خلال هذا التابع كيف ستملؤه
- GridBagConstraints.NONE : لا يتم ملء المكون (المكون سيبقى حجمه كما هو)
- GridBagConstraints.HORIZONTAL : يتم ملء المكون فقط في الاتجاه الأفقي
- GridBagConstraints.VERTICAL : يتم ملء المكون فقط في الاتجاه العمودي
- GridBagConstraints.BOTH : يتم ملء المكون في كلا الاتجاهين (الأفقي والعمودي)
- Insets الفراغ بين العنصر و حواف الخلية
- anchor في حال كان العنصر اصغر من حجم الخلية التي سيشغلها نحدد من خلال هذا التابع كيف سيتوضع العنصر
- GridBagConstraints.NORTH
- GridBagConstraints.SOUTH
- GridBagConstraints.EAST
- GridBagConstraints.WEST
- GridBagConstraints.CENTER

مثال:

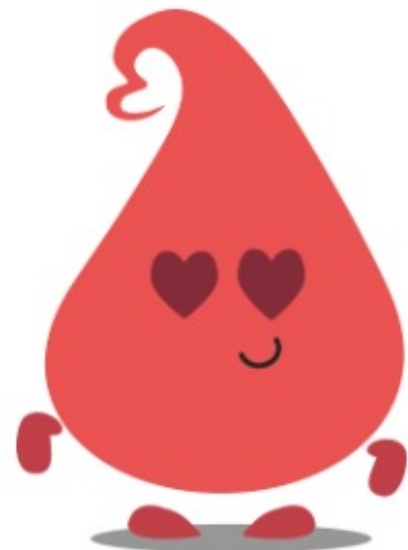
```
import javax.swing.*;
import java.awt.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("GridBagLayout Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
// Layout Manager ك GridBagLayout تحديد
frame.setLayout(new GridBagLayout());
// إنشاء كائن GridBagConstraints للتحكم في ترتيب
المكونات ضمن الشبكة
GridBagConstraints gbc = new
GridBagConstraints();
// إنشاء بعض الأزرار كمكونات داخل Frame
JButton button1 = new JButton("Button 1");
JButton button2 = new JButton("Button 2");
JButton button3 = new JButton("Button 3");
JButton button4 = new JButton("Button 4");
// الخلية في الأول الزر وضع (0, 0)
gbc.gridx = 0; // الأول العمود
gbc.gridy = 0; // الأول الصف
// زر 1 سيأخذ عرض عمودين
gbc.gridwidth = 2;
// ملء المساحة افقيا
gbc.fill = GridBagConstraints.HORIZONTAL;
frame.add(button1, gbc);
// وضع الزر الثاني في الخلية (0, 1)
gbc.gridx = 0;
gbc.gridy = 1;
// زر 2 سيأخذ عمود واحد فقط
gbc.gridwidth = 1;
// لا يتم ملء المساحة
gbc.fill = GridBagConstraints.NONE;
```

```
frame.add(button2, gbc);
// وضع الزر الثالث في الخلية (0, 2)
gbc.gridx = 0;
gbc.gridy = 2;
gbc.gridwidth = 1;
gbc.fill =
GridBagConstraints.HORIZONTAL;
frame.add(button3, gbc);
// وضع الزر الرابع في الخلية (1, 2)
gbc.gridx = 1;
gbc.gridy = 2;
frame.add(button4, gbc);
// frame size
frame.setSize(400, 200);
// frame visibility
frame.setVisible(true); }
```



Excellence is to do a common thing
in an uncommon way.



box layout : الذي يجعل العناصر تتوضع ضمن الحاوية كأنها صندوق سواء عرضي أو طولي حيث توضع العناصر فوق بعضها أو بجانب بعضها يمكن وضع مسافات بأشكال مختلفة بين العناصر ضمن

يمكن التعامل معه على النحو التالي :

```
import javax.swing.*;
import java.awt.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("BoxLayout Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel panel = new JPanel();
        // عمل صندوق بشكل طولي من خلال جعله يتعامل مع محور الترتيب
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        // عمل صندوق بشكل عرضي من خلال جعله يتعامل مع محور الفواصل
        panel.setLayout(new BoxLayout(panel, BoxLayout.X_AXIS));
        // اضافة زر
        panel.add(new JButton("Button 1"));
        // لاضافة مسافة عرضية
        panel.add(Box.createHorizontalStrut(20));
        // لاضافة مسافة طولية
        panel.add(Box.createVerticalStrut(20));
        // اضافة زر ثاني
        panel.add(new JButton("Button 2"));
        panel.add(Box.createHorizontalGlue());
        // اضافة مسافة مرنة عرضية
        panel.add(Box.createHorizontalGlue());
        // اضافة مسافة مرنة طولية
        panel.add(Box.createVerticalGlue());
        // اضافة زر ثالث
        panel.add(new JButton("Button 3"));
        // اضافة مسافة بشكل عام مع تحديد بعد المسافة بالناحيتين
        panel.add(Box.createRigidArea(new Dimension(50, 40)));
        // اضافة زر رابع
        panel.add(new JButton("Button 4"));
        // اضافة الحاوية الصندوق
        frame.add(panel);
        frame.setSize(300, 300);
        frame.setVisible(true); } }
```

علماً أن اختيار توابع وضع المسافة مبنية على الصندوق الذي اخترناه، أي create vertical glue لن تعمل في حال كان الصندوق افقياً و العكس صحيح، فقط وضعنا التوابع لعرضها.

آخر layout سنتحدث عنها هي :

■ الـ card layout، الحاوية التي يتم تنسيقها من خلال هذه الـ layout يمكن وضع عناصر داخلها بشكل طبقات فوق بعضها البعض بحيث نظهر ما نريده منها من خلال عدة توابع نستخدمه بالشكل التالي :

```
import javax.swing.*;
import java.awt.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("CardLayout
        Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_O
        N_CLOSE);
        // نحفظ به بشكل مستقل لأننا نريد أن نتعامل معها
        CardLayout cardLayout = new CardLayout();
        // نعمل حاوية نمرر لها ال layout
        JPanel cardPanel = new JPanel(cardLayout);
        // نضيف عناصر للحاوية من خلال تمرير العنصر
        // إضافة ل string يمثل اسم هذا ال card بحيث نستطيع
        الوصول له لاحقاً من خلال اسمه
        cardPanel.add(new JButton("Card 1"), "Card1");
        cardPanel.add(new JButton("Card 2"), "Card2");
        cardPanel.add(new JButton("Card 3"), "Card3");
        // نضيف الحاوية للنافذة
        frame.add(cardPanel, BorderLayout.CENTER);
        // هذه الحاوية لنضع فيها الأزرار لنرى أثر التوابع
        // التي يمكن أن نستخدمها على هذه ال layout
        JPanel buttonPanel = new JPanel();
        // زر للتحرك للعنصر التالي ضمن ال cards
        JButton nextButton = new JButton("Next");
        // زر للتحرك للعنصر السابق ضمن ال cards
        JButton prevButton = new JButton("Previous");
```

```
        buttonPanel.add(prevButton);
        buttonPanel.add(nextButton);
        frame.add(buttonPanel,
        BorderLayout.SOUTH);
        التابع يظهر العنصر ضمن الحاوية // ذو
        الاسم Card1 على الواجهة
        cardLayout.show(cardPanel, "Card1");
        سيتم شرح معنى add action listener
        بالفقرة الجاي لكن ببساطة عند
        كبس الزر سيقوم // باستدعاء التوابع التي
        نريد اختبارها
        nextButton.addActionListener(
        التابع الذي يعرض العنصر التالي
        ضمن ال layout
        e -> cardLayout.next(cardPanel));
        prevButton.addActionListener(
        التابع الذي يعرض العنصر السابق
        ضمن ال layout
        e -> cardLayout.previous(cardPanel));
        frame.setSize(400, 300);
        frame.setVisible(true); }}
```



Event listeners

عندما نتعامل مع العناصر في swing فنحن غالباً نود التفاعل معها و جعلها تقوم بأشياء معينة عند هذا التفاعل و لتحقيق هذا الأمر يوجد لدينا أمر يسمى Listeners و التي هي كائنات تحقق استجابة للتفاعلات بطرق نحددها نحن كمبرمجين.

- WINDOWLISTENER
- ACTIONLISTENER
- LISTSELECTIONLISTENER
- KEYLISTENER
- CHANGE-LISTENER
- MOUSELISTENER
- MOUSEMOTIONLISTENER
- MOUSEWHEELLISTENER

سنشرح بعضها:

1. ActionListener:

الذي هو الشكل البسيط من ال Listeners التي يمكن إضافته للعناصر القابلة للتفاعل مثل ال JBUTTONS و ال JRadioButtons وال JCheckBox وال JTextField و ال JPasswordField و غيره من العناصر التي هي بطبيعتها قابلة للتفاعل .. ببساطة شديدة عند التفاعل مع المكون يتم استدعاء التابع

مثال:

```
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("ActionListener Example");

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 200);
        // زر للتجربة
        JButton button = new JButton("Click Me");
        // نضيف له طريقة للتفاعل من خلال التابع التالي
        button.addActionListener(
```

حيث ال action event هو object عند القيام بالتفاعل مع العنصر يرجع تفاصيل هذا التفاعل الحاصل، مثلاً ما هو سبب حدوث التفاعل من خلال التابع getSource، لكن لن نستفيد منه كثيراً الآن سنرى فائدته تالياً



```

new ActionListener() {
    // نقوم باوفر رايد لهذا التابع
    @Override
    public void actionPerformed(ActionEvent e) {
        // عند حصول تفاعل مع الزر نفذ ما ضمن هذا التابع
        System.out.println("Button was clicked!");
    }
});
// الزر اضافة
frame.add(button);
frame.setVisible(true);
}
}

```

2. mouse listener : يستجيب للتفاعلات التي يمكن للماوس إحداثها

- void mouseClicked(MouseEvent e) ; ضغط و افلات زر بالماوس
- void mousePressed(MouseEvent e); ضغط زر بالماوس
- void mouseReleased(MouseEvent e); افلات زر بالماوس
- void mouseEntered(MouseEvent e); دخول مؤشر الماوس للمنطقة
- void mouseExited(MouseEvent e); خروج مؤشر الماوس من المنطقة عند حدوث أي من الاحداث السابقة يمكن تنفيذ امر معين

ملاحظة: بدلا من استخدام كافة التوابع السابقة يمكن استخدام أمر يسمى mouse adapter الذي يؤمن لك mouse listener مع implementation فارغة للتوابع التي لا نريد استعمالها مما ذكرناه سابقاً مما لا يجبرنا على عمل override لها يدوياً.

فمثلاً انا اريد فقط القيام بأمر في حالة دخول مؤشر الماوس للمنطقة لا داعي لأقوم بعمل object من mouse listener و أقوم بعمل توابع فارغة يدوياً أقوم بعمل mouse adapter تعمل override للتوابع التي نرغبها فقط

```

import java.awt.event.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("MouseListener Example");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JLabel label = new JLabel("Perform a mouse action!", SwingConstants.CENTER);
        frame.add(label);

        // اضافة ال mouse listener للنافذة
        frame.addMouseListener(new MouseListener() {
            @Override
            public void mouseClicked(MouseEvent e) {
                // getx and gety يجلبان احداثيات مكان المؤشر على محوري الوايات و الاكسات لحظة الضغط
                label.setText("Mouse clicked at: (" + e.getX() + ", " + e.getY() + ")");
            }
            @Override
            public void mousePressed(MouseEvent e) {
                label.setText("Mouse pressed!");
            }
            @Override
            public void mouseReleased(MouseEvent e) {
                label.setText("Mouse released!");
            }
            @Override
            public void mouseEntered(MouseEvent e) {
                label.setText("Mouse entered the frame!");
            }
            @Override
            public void mouseExited(MouseEvent e) {
                label.setText("Mouse exited the frame!");
            }
        });

        // مثال على ال adapter لكن لن اضيفه لشيء فقط لاعرض كيفية كتابته
        MouseAdapter mouseAdapter = new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
            }
        };
    }
};

```

كما رأينا التوابع e.getx() and e.gety() قدمت لنا معلومات مفيدة عن سبب الحدث مثل احداثيات مكان مؤشر
الماوس على الشاشة

3. window listener : يستجيب للتفاعلات التي نقوم بها مع النافذة

- void windowOpened(WindowEvent e); عند ظهور النافذة لأول مرة
- void windowClosing(WindowEvent e); عند محاولة اغلاق النافذة
- void windowClosed(WindowEvent e); عندما يتم اغلاق النافذة
- void windowIconified(WindowEvent e); عند عمل minimize للنافذة
- void windowDeiconified(WindowEvent e); عند إعادة النافذة من حالة minimization
- void windowActivated(WindowEvent e); لما تصبح النافذة هي الواجهة التي تتعامل معها ضمن النظام حالياً؛
- void windowDeactivated(WindowEvent e); لما تصبح النافذة بالخلفية ولا تتعامل معها ضمن النظام حالياً

```
import javax.swing.*;
import java.awt.event.*;
public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("WindowListener Example");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        JLabel label = new JLabel("Window Events will be displayed here.",
        SwingConstants.CENTER);
        frame.add(label);
        // نضيف لل frame هذا ال listener
        frame.addWindowListener(new WindowListener() {
            @Override
            public void windowOpened(WindowEvent e) {
                System.out.println("Window opened.");
            }
            @Override
            public void windowClosing(WindowEvent e) {
                // اخراج نافذة تأكيد عند الضغط على x لمحاولة اغلاق البرنامج
                int confirm = JOptionPane.showConfirmDialog(
                    frame,
                    "Are you sure you want to exit?",
```

```

        "Confirm Exit",
        JOptionPane.YES_NO_OPTION
    );
    if (confirm == JOptionPane.YES_OPTION) {
        // النافذة يغلق
        frame.dispose();
    }
}
}
@Override
public void windowClosed(WindowEvent e) {
    System.out.println("Window closed.");
}
@Override
public void windowIconified(WindowEvent e) {
    System.out.println("Window minimized.");
}
@Override
public void windowDeiconified(WindowEvent e) {
    System.out.println("Window restored.");
}
@Override
public void windowActivated(WindowEvent e) {
    System.out.println("Window activated.");
}
@Override
public void windowDeactivated(WindowEvent e) {
    System.out.println("Window deactivated.");
}
});
frame.setVisible(true);
}
}

```

كذلك هنالك window adapter يشابه سابقه في ال mousetlistener يقوم بنفس الأمر

آخر Listener سنشرحه هو:

4. `KeyListener` : يستجيب للتفاعلات التي تتم من خلال الكيبورد يعمل على العناصر التي يمكن الكتابة عليها (has a focus) مثل ال `TextField` و ال `PasswordField` و حتى ال panels (استدعاء تابع `setFocusable(true)`)

`void keyPressed(KeyEvent e)` : نضغط أي زر من لوحة المفاتيح

`void keyReleased(KeyEvent e)` : نحرر أي زر من لوحة المفاتيح

`void keyTyped(KeyEvent e)` : عند ضغط و افلات زر ليكتب حرف (تعمل فقط في حال كان الزر المضغوط ضمن لوحة المفاتيح يكتب حرف)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class Main {
    private int x = 150, y = 150; // Position of a rectangle
    public Main() {
        JFrame frame = new JFrame("Game Controls Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // anonymous class لعمل بائيل بمواصفات محددة من خلال
        JPanel panel = new JPanel() {
            @Override
            // هذا التابع هو التابع الذي يرسم البانلز بالعادة
            protected void paintComponent(Graphics g) {
                // نستدعي التابع الذي يرسم المكونات بالكلاس الأب
                super.paintComponent(g);
                // نحدد لون الرسم ليكون أزرقاً
                g.setColor(Color.BLUE);
                // نرسم مستطيل بأحداثيات x,y و بعرض 50 و طول 50
                g.fillRect(x, y, 50, 50);
            }
        };
        // نحدد حجم مفضل ليكون
        panel.setPreferredSize(new Dimension(400, 300));
        // تابع يجعل البانل قابلة للتركيز عليها (focus)
        panel.setFocusable(true);
        // اضافة ال key listener
        panel.addKeyListener(new KeyListener() {
```

```

@Override
public void keyPressed(KeyEvent e) {
    //e.getKeyCode
    //يرجع رقم الزر الذي تم ضغطه بصياغة موضوعة بشكل مسبق
    switch (e.getKeyCode()) {
        //لو ضغط سهم للاعلى
        case KeyEvent.VK_UP -> y -= 10;
        //لو ضغط زر للاسفل
        case KeyEvent.VK_DOWN -> y += 10;
        //لو ضغط زر اليسار
        case KeyEvent.VK_LEFT -> x -= 10;
        //لو ضغط زر اليمين
        case KeyEvent.VK_RIGHT -> x += 10;
        //الخلاصة يعدل احداثيات المربع
        //اعد رسم المربع
        panel.repaint();
    }
    @Override
    public void keyReleased(KeyEvent e) {
    }
    @Override
    public void keyTyped(KeyEvent e) {
    }
    });
    //نضيف البانل للنافذة
    frame.add(panel);
    //يجعل النافذة تأخذ حجماً يتناسب مع محتوياتها الداخلية
    frame.pack();
    frame.setVisible(true);}
    public static void main(String[] args) {
        //إنشاء كائن من main
        new Main();
    }

```

*يوجد أيضاً keyAdapter لنفس الغايات لسابقه.

■ ملاحظة : التابع repaint عند استدعائه يعيد استدعاء paintComponent بالتالي هذا سبب تغير مكان المربع.

■ ملاحظة 2 : استخدام الأسهم ضمن switch statement يعبر عن طريقة موجودة بجافا لكتابة ال cases بشكل اسلس.

حيث تقوم بتنفيذ ال case و الخروج من ال switch يمكن أيضا استخدام هذه الطريقة جعل السويتش تقوم بإرجاع قيمة كما التالي:

مثال:

```
int day = 3;
String dayType = switch (day) {
    case 1 -> {
        System.out.println("Start of the week");
        yield "Monday";
    }
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    default -> "Invalid day";
};
```

* عوضاً عن اسناد القيمة يدوياً.

و سنكتفي بهذا القدر من شرح ال listeners، يوجد العديد و العديد من ال listeners غير التي شرحناها و التي لها استخدامات مختلفة و لكنها تقوم على نفس المبدأ .

✎ و الآن بعد أن انتهينا من الفقرات الأساسية ضمن swing نذكر بعض النقاط المهمة و مثلاً عملياً على ما يمكن القيام به

يمكن خلق classes ترث من المكونات الخاصة ب swing و وضع خصائص لها حسب الرغبة على سبيل المثال سأعيد تكوين المربع الذي يتحرك سابقاً من خلال جعل ال class نفسه هو panel

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class Main extends JPanel implements KeyListener {
    private int x = 150, y = 150;
    public Main() {
        setPreferredSize(new Dimension(400, 300));
        setFocusable(true);
        addKeyListener(this);
    }
    repaint();
}
@Override
public void keyReleased(KeyEvent e) {
}
@Override
public void keyTyped(KeyEvent e) {
}
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    g.setColor(Color.BLUE);
    g.fillRect(x, y, 50, 50);
}
public static void main(String[] args) {
    JFrame frame = new JFrame("Game Controls Example");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.add(new Main());
    frame.pack();
    frame.setVisible(true);
}
}

```

الآن أصبح ال main class هو ذاته ال panel التي تحوي المربع الذي يتحرك، و بدلا من عمل بائل و التعديل عليها في كل مرة ارغب بانشاء امر مشابه أقوم فقط بعمل اوبجكت منها، و هذا لب البناء من خلال swingL حيث نقوم بتكوين الواجهة بكلاس مستقل مثلاً وفقط عمل كائن منها عند الرغبة باضافتها للنافذة و هذا ينطبق على كافة المكونات الأخرى .

الآن فقط سنقوم بتقديم مثال عملي على شيء يمكن القيام به من خلال ما تعلمناه سابقاً و بشكل مبسط من خلال نافذة بسيطة لتسجيل الدخول.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
public class Main extends JFrame implements ActionListener {
// تعريف المكونات التي نحتاجها لشاشة تسجيل دخول
    private JTextField usernameField;
// الباني
    public Main() {
        setTitle("Login Form");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// استخدام GridLayout لترتيب المكونات
// 4 صفوف و 2 أعمدة مع تباعد 10
        setLayout(new GridLayout(4, 2, 10, 10));
// إنشاء كلمة "الاسم"
        JLabel usernameLabel = new JLabel("Username:");
// إنشاء حقل الاسم
        usernameField = new JTextField();
// إنشاء كلمة "كلمة المرور"
        JLabel passwordLabel = new JLabel("Password:");
// إنشاء حقل كلمة المرور
        passwordField = new JPasswordField();
// إنشاء الزر الخاص بتسجيل الدخول
        loginButton = new JButton("Login");
// سنظهر النتيجة على هذا الlabel
        messageLabel = new JLabel("");
// إضافة ال ActionListener على الزر
        loginButton.addActionListener(this);
// إضافة المكونات إلى ال frame
        add(usernameLabel);
        add(usernameField);
        add(passwordLabel);
        add(passwordField);
        add(loginButton);
        add(messageLabel);
    }
}
```

```
// هنا نجعل موقع ال frame بالنسبة لnull (أي مستقلاً) مما يظهره في منتصف الشاشة
setLocationRelativeTo(null);
setVisible(true);}

// القيام ب override للتابع actionPerformed بحكم أن ال main class يعتبر instance
كذلك من ActionListener
@Override
// عند حدوث ضغط على الزر أعلاه سيتم تنفيذ التالي
public void actionPerformed(ActionEvent e) {
// نأخذ الاسم من ال usernameField
String username = usernameField.getText();
// نأخذ الاسم من ال passwordField
String password = new String(passwordField.getPassword());
// الآن سنقارن باسم و كلمة سر افتراضيتين admin و 1234
if (username.equals("admin") && password.equals("1234")) {
// لو ادخلهما بشكل صحيح نظهر له أن تسجيل الدخول تم بنجاح على ال label النتيجة
messageLabel.setForeground(Color.GREEN);
messageLabel.setText("Login Successful!");
} else {
// لو ادخلهما بشكل خاطئ نظهر له أن تسجيل الدخول فشل على ال label النتيجة
messageLabel.setForeground(Color.RED);
messageLabel.setText("Invalid Username or Password");
}
}

public static void main(String[] args) {
// إنشاء نافذة تسجيل الدخول
new Main();}}
```



The end